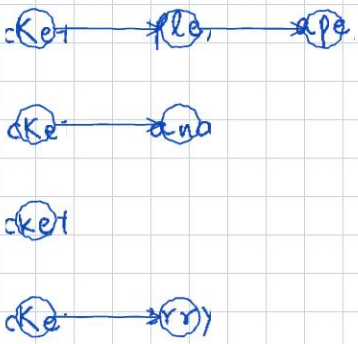
Hash Tables

A hash table is a data structure that maps keys to values for highly efficient lookup. It uses a hash function to compute an index into an array of buckets, where the desired value is stored.

The core idea: instead of searching through a structure, we compute exactly where the data should be!

Collisions happen when two different keys hash to the same index. The most common resolution strategy is Separate Chaining. In this approach, each bucket holds a linked list of all key-value pairs that hash to that index.

Let's look at an array of buckets with linked lists. If we insert ("apple", 5) and ("grape", 9) and both hash to index 0, they form a chain:



Operations and Complexity (Average Case):
- insert(key, val): Compute index → prepend to linked list → O(1)
- search(key): Compute index → traverse short list → O(1)
- delete(key): compute index → remove from list → O(1)

Load Factor and Rehashing:
The load factor α measures how full the hash table is:
$\alpha = \frac{n}{m}$ (where n = number of elements, m = number of buckets)

If α gets too high (e.g., > 0.75), collisions increase, degrading performance. To fix this, we Rehash:
1. Allocate a new array of size $2m$.
2. Iterate through all existing elements.
3. Re-insert them into the new array using the new hash function.

Comparison with Arrays and BSTs:
- Arrays: O(1) access by index, but O(n) search by value.
- Binary Search Trees: O(log n) search, insert, delete (if balanced), and maintains sorted order.
- Hash Tables: O(1) average for search, insert, and delete, but does NOT maintain any ordering among elements.

Hash tables = O(1) average, but worst-case O(n)